

CS 351 -> Roommate Finder App

By: Jash, Julia, Jason

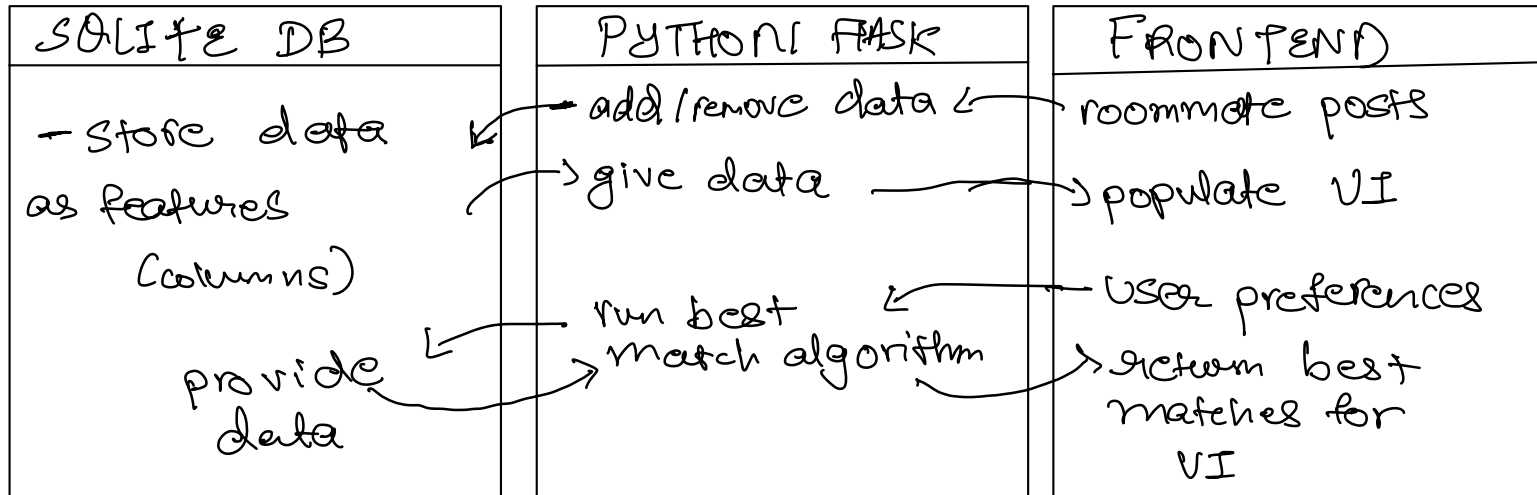
Rough Sketch

Problem -> Many students at UIC struggle to find roommates through random facebook groups, chats, etc.

Solution -> create a webApp that makes roommate finding easier. Instead of manually searching user can filter posts & can also put their preferences to find a best match. User preferences we are using:-

Name, Smoking, Drinking, Description, Country, language, Sex, Price Range, Subleasing, Time period, Email, Phone, Social Media, Religion !!

Idea in a glance



Let's go in more detail:

Databases we are going to use SQLite to design our own database.

The database schema will be

Each preference as a feature

ID

First name	LAST Name	SMOKING	DRINKING	RENT MAX	COUNTRY

Data will be pulled by python code called using GET requests by frontend.

User → Input preferences → Call API → data cleaning & testing
↓
database

Data Cleaning & Testing

↳ Each Column will have constraints which will be checked before adding data to DB. Example Smoking can only be a boolean value (0/1) while rent has to be an integer between (300 - 7000), etc

This cleaning will be done in frontend & backend to ensure strict data types & uniform data to work with.

DATA STRUCTURES & FEATURES

→ Each feature is a data structure pretty much.

Search using Trie

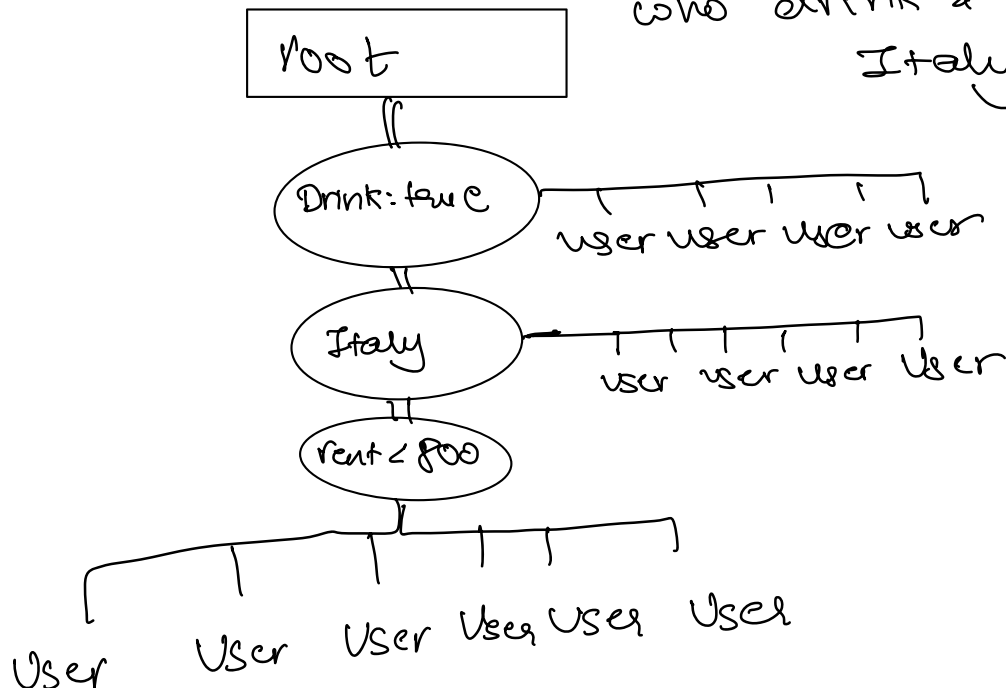
Filters:

Smoking ☐ Rent ☐ Sex ☐ Sublease ☐

Drinking ☐ Country ☐ Language ☐

↳ returns all filters

Trie → For each filter we get all users for that feature. ex filter all people who drink & are from Italy & rent < 800



The trie is loaded at start of app and then used for filter.

2. Find Best Matches using priority Queue

Use trie to get profiles & assign score based on matches.

If user preference is no drinking or smoking we get all profiles with those preferences and we run a scoring algorithm. & return best matches!!

Time

Space

Trie Insert	$O(nm)$	$O(nm)$	$n \rightarrow \text{roommate posts}$ $m \rightarrow \text{attributes}$
Trie Search	$O(m+k)$	$O(nm)$	
PQ Insert	$O(k \log k)$	$O(k)$	$k = \text{no of matches}$ $k = \text{size of heap}$
PQ Extract	$O(p \log p)$	$O(k)$	
Over ALL	$O(nm + k \log k)$	$O(nm + k)$	$p = \text{matches}$

why PriorityQueue $\rightarrow$ Heap is much faster at $O(n \log n)$ ~~is~~ than a linked list which we were going to use at $O(n)$

why Trie $\rightarrow$ Best for filtering compared to Hashmap which have fast lookup but bad filtering since we need to scan everything.

Timeline

No	Task	Start Date	Due Date	Status	Group Members
	1 FGP3	Feb 27th	Feb 27th	Completed	All
	2 FGP4	March 3rd	March 20th	Inprogress	All
3	Finalize UI Layout & Design	March 20th	March 27th	Inprogress	Julia, Jason
4	Create DB & perform basic CRUD operation	March 20th	March 27th	Inprogress	Jash
5	Create backend API	March 27th	March 29th	-	Jash
6	Create Frontend base Layout with backend API	March 29th	April 3rd	-	Julia, Jason
	7 FGP5	April 3rd	April 3rd	-	All
8	Create best match algorithm API on backend	April 4th	April 11th	-	Jash
9	Create UI for best match algorithm API	April 11th	April 23rd	-	Julia, Jason
	10 FGP6	April 24th	April 24th	-	All
11	Test UI & backend	April 24th	April 26th	-	All
12	Create presentation	April 24th	April 28th	-	All
	13 FGP7	April 29th	April 29th	-	All